

UNIVERSIDAD
SERGIO ARBOLEDA

Diseño e implementación de una gramática LL(1)

Un lenguaje con operaciones aritméticas, funciones trigonométricas y asignación de variables

Lenguajes de programación y traducción

David Andrés Castellanos Angulo

Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 28 de septiembre de 2026

1. Objetivo del trabajo

El trabajo pide diseñar e implementar una gramática LL(1) para un lenguaje de programación que soporte `+`, `-`, `*`, `/`, `%` y el valor absoluto, las funciones `sin`, `cos` y `tan`, y la asignación de variables. Hay que garantizar las tres fases de análisis, mostrar los conjuntos PRIMEROS, SIGUIENTES y de predicción, y hacer pruebas de implementación.

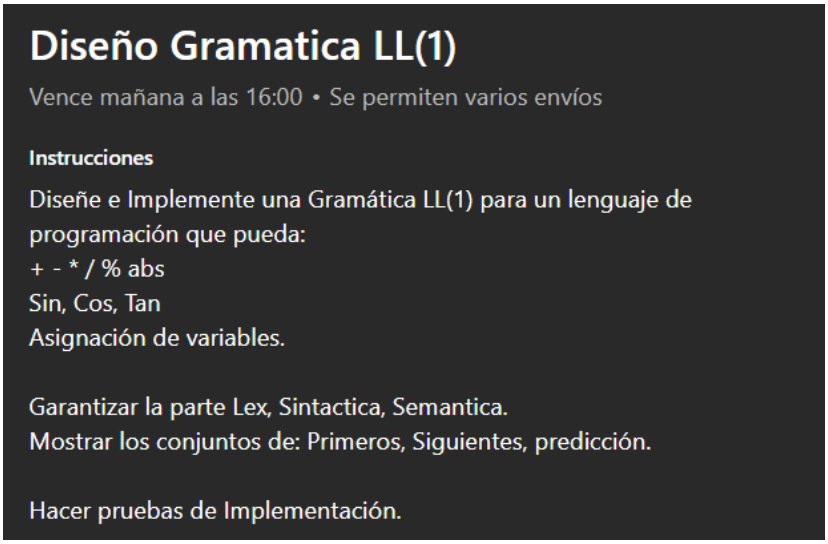


Figura 1: Enunciado del trabajo.

Lo hice en Python, sin librerías externas, siguiendo el orden que vimos en clase para construir un analizador descendente recursivo: primero la gramática, después los conjuntos, después la verificación LL(1) y solo al final el código.

2. De la gramática natural a la gramática LL(1)

No empecé escribiendo la gramática definitiva. La escribí como se lee en matemáticas y después le apliqué las transformaciones, que es donde está el trabajo de verdad.

2.1. Primer borrador: recursión izquierda

Listing 1: `grammars/borrador1_recursion.txt`: la versión natural.

```
# Primer borrador: escribí la gramática como se lee en matemáticas.
# Tiene recursión izquierda en expr y en term, así que NO sirve para
    un ASDR.

prog     -> sents
sents    -> sent sents | ε
sent     -> ID asig expr fin

expr     -> expr mas term | expr menos term | term
term     -> term por fact | term entre fact | term modulo fact | fact

fact     -> menos fact | unidad
unidad   -> NUM | ID | abre expr cierra | barra expr barra | func abre
     expr cierra
func     -> abs | sin | cos | tan
```

La regla *expr* → *expr* + *term* dice exactamente lo que uno quiere: una suma es una suma seguida de un término. El problema es que tiene **recursión izquierda**, y eso rompe el descenso recursivo: la función `expr()` se llamaría a sí misma como primera acción, sin haber consumido ningún token, y quedaría en un ciclo infinito.

Se ve también en los conjuntos. Le pasé este borrador al programa y reporta el conflicto:

```
¿ES LL(1)? NO. Reglas con predicción en común:
  expr: reglas 5 y 6 comparten { ID, menos, NUM, abre, barra, abs, sin,
    cos, tan }
  expr: reglas 5 y 7 comparten { ID, menos, NUM, abre, barra, abs, sin,
    cos, tan }
  expr: reglas 6 y 7 comparten { ID, menos, NUM, abre, barra, abs, sin,
    cos, tan }
  term: reglas 8 y 9 comparten { ID, menos, NUM, abre, barra, abs, sin,
    cos, tan }
```

Las tres reglas de *expr* predicen los mismos tokens, así que con un token de anticipación no hay forma de escoger.

2.2. Segundo borrador: prefijo común

Lo primero que intenté fue pasar la recursión al otro lado:

```
expr -> term mas expr | term menos expr | term
```

Ya no hay recursión izquierda, pero aparece el otro obstáculo: las tres alternativas empiezan por *term*, o sea, tienen **prefijo común**. El analizador tendría que consumir todo un término antes de saber cuál regla escogió, y eso es justamente lo que no se puede hacer con un solo token. El programa vuelve a reportar conflicto en *expr* y en *term* (`grammars/borrador2_prefijo.txt`).

2.3. Las dos transformaciones

La regla general para quitar la recursión izquierda inmediata es:

$$A \to A\alpha \mid \beta \quad \Longrightarrow \quad A \to \beta A', \qquad A' \to \alpha A' \mid \varepsilon$$

Aplicada a *expr*, con $\alpha_1 =$ `+` *term*, $\alpha_2 =$ `-` *term* y $\beta =$ *term*:

$$expr \to term\ expr', \qquad expr' \to \texttt{+}\ term\ expr' \mid \texttt{-}\ term\ expr' \mid \varepsilon$$

Y lo mismo en *term* con `*`, `/` y `%`. La otra transformación, factorizar por la izquierda,

$$A \to \alpha\beta_1 \mid \alpha\beta_2 \quad \Longrightarrow \quad A \to \alpha A', \qquad A' \to \beta_1 \mid \beta_2$$

lleva al mismo resultado si se parte del segundo borrador: se saca el *term* común y lo que queda detrás pasa a un no terminal nuevo. Por eso los dos caminos terminan en la misma gramática.

En el archivo los llamé `expr_r` y `term_r`, porque el apóstrofo no es cómodo en nombres de archivo y de funciones; en el informe los escribo *expr'* y *term'*.

2.4. Gramática final

Listing 2: `grammars/lenguaje.txt`.

```
# Gramática LL(1) del lenguaje
# Los no terminales son los símbolos de la izquierda; el resto son
    terminales.
# Los terminales en MAYÚSCULA son clases de token que entrega el
    analizador léxico.

prog     -> sents
sents    -> sent sents | ε
sent     -> ID asig expr fin

expr     -> term expr_r
expr_r   -> mas term expr_r | menos term expr_r | ε

term     -> fact term_r
term_r   -> por fact term_r | entre fact term_r | modulo fact term_r |
    ε

fact     -> menos fact | unidad

unidad   -> NUM
unidad   -> ID
unidad   -> abre expr cierra
unidad   -> barra expr barra
unidad   -> func abre expr cierra

func     -> abs | sin | cos | tan
```

Los terminales no son los símbolos sueltos sino **clases de token**: `ID` para cualquier nombre de variable y `NUM` para cualquier número, porque la gramática no puede enumerarlos uno por uno.

Tres cosas quedaron resueltas de una vez:

- **Precedencia.** *expr* agrupa sumas y restas, *term* agrupa producto, división y módulo, y *fact* queda más abajo. Como lo que está más abajo queda más profundo en el árbol, se evalúa primero: `2 + 3 * 4` da 14.
- **Nada de recursión izquierda**, por la transformación anterior.
- **Nada de prefijos comunes.** Las cinco alternativas de *unidad* empiezan cada una con un token distinto, y agrupé las cuatro funciones en *func* para que *unidad* tenga una sola alternativa que empiece por función.

3. Conjuntos y verificación LL(1)

Como las transformaciones cambian la gramática, los conjuntos hay que **recalcularlos** sobre la versión final: el hecho de haber quitado la recursión izquierda no garantiza por sí solo que la gramática sea LL(1). Los calcula `conjuntos.py`, el mismo programa del taller anterior, con algoritmos de punto fijo.

Para cada regla, la predicción se obtiene así:

$$\mathrm{PRED}(A \to \alpha) = \big(\mathrm{PRIMEROS}(\alpha) - \{\varepsilon\}\big) \cup \begin{cases} \mathrm{SIGUIENTES}(A), & \text{si } \varepsilon \in \mathrm{PRIMEROS}(\alpha), \\ \varnothing, & \text{si no.} \end{cases}$$

Cuadro 1: PRIMEROS y SIGUIENTES de la gramática final.

No terminal	PRIMEROS	SIGUIENTES
prog	{ID, ε}	{$}
sents	{ID, ε}	{$}
sent	{ID}	{ID, $}
expr	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}	{fin, cierra, barra}
expr_r	{mas, menos, ε}	{fin, cierra, barra}
term	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}	{fin, mas, menos, cierra, barra}
term_r	{por, entre, modulo, ε}	{fin, mas, menos, cierra, barra}
fact	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}	{fin, mas, menos, por, entre, modulo, cierra, barra}
unidad	{ID, NUM, abre, barra, abs, sin, cos, tan}	{fin, mas, menos, por, entre, modulo, cierra, barra}
func	{abs, sin, cos, tan}	{abre}

Cuadro 2: Conjuntos de predicción de las 24 reglas.

#	Regla	PREDICCIÓN
1	*prog* → *sents*	{ID, $}
2	*sents* → *sent sents*	{ID}
3	*sents* → ε	{$}
4	*sent* → ID asig *expr* fin	{ID}
5	*expr* → *term expr_r*	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}
6	*expr_r* → mas *term expr_r*	{mas}
7	*expr_r* → menos *term expr_r*	{menos}
8	*expr_r* → ε	{fin, cierra, barra}
9	*term* → *fact term_r*	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}
10	*term_r* → por *fact term_r*	{por}
11	*term_r* → entre *fact term_r*	{entre}
12	*term_r* → modulo *fact term_r*	{modulo}
13	*term_r* → ε	{fin, mas, menos, cierra, barra}
14	*fact* → menos *fact*	{menos}
15	*fact* → *unidad*	{ID, NUM, abre, barra, abs, sin, cos, tan}
16	*unidad* → NUM	{NUM}
17	*unidad* → ID	{ID}
18	*unidad* → abre *expr* cierra	{abre}
19	*unidad* → barra *expr* barra	{barra}
20	*unidad* → *func* abre *expr* cierra	{abs, sin, cos, tan}
21	*func* → abs	{abs}
22	*func* → sin	{sin}
23	*func* → cos	{cos}
24	*func* → tan	{tan}

La condición LL(1) es que, para dos reglas distintas del mismo no terminal,

$$\mathrm{PRED}(A \to \alpha) \cap \mathrm{PRED}(A \to \beta) = \varnothing.$$

El programa compara los pares regla por regla y confirma que se cumple:

```
¿ES LL(1)? SÍ: las reglas de cada no terminal tienen predicciones
   disjuntas.
```

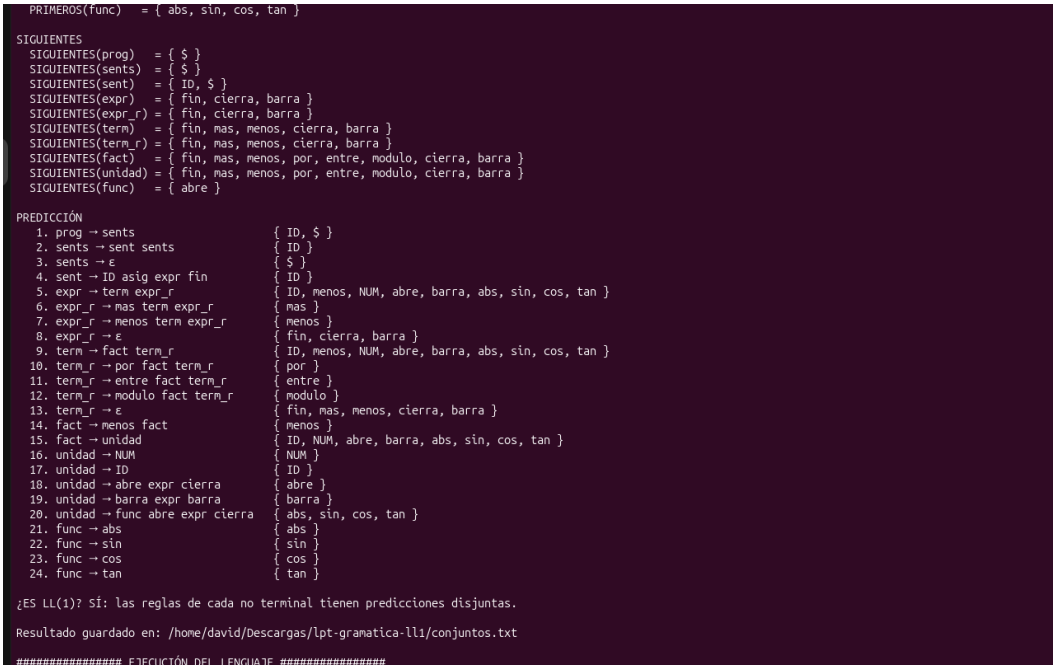


Figura 2: Conjuntos SIGUIENTES y de predicción de la gramática final, con la verificación LL(1).

Se puede revisar a simple vista en la tabla: las tres reglas de *expr'* predicen `mas`, `menos` y {`fin`, `cierra`, `barra`}; y las cinco de *unidad* predicen cada una un token distinto. El caso de *expr'* $\rightarrow \varepsilon$ es el que usa SIGUIENTES: como la regla puede desaparecer, se predice con los tokens que pueden venir después.

Vale la pena aclarar algo que vimos en clase: un conflicto LL(1) no significa que la gramática sea ambigua. Los dos borradores de la sección anterior generan el mismo lenguaje que la versión final y no son ambiguos; simplemente no se pueden analizar con un solo token de anticipación. LL(1) implica que no hay ambigüedad, pero al revés no.

4. Construcción del analizador

Seguí los cinco pasos del ASDR: definir la gramática y su símbolo inicial, calcular PRIMEROS, SIGUIENTES y predicción, verificar que las predicciones sean disjuntas, escribir una función por no terminal con una operación `emparejar` para los terminales, y finalmente tomar el primer token, llamar al símbolo inicial y comprobar el fin de entrada.

4.1. Análisis léxico

`lexico.py` recorre el texto y arma la lista de tokens. Reconoce enteros y reales, identificadores, las palabras reservadas, los símbolos y los comentarios con `#`. Cada token guarda su línea y columna, que es lo que permite reportar los errores con la posición exacta:

```
Error léxico: línea 2, columna 7: símbolo no reconocido '@'.
```

Al final agrega el token `$`, que es el marcador de fin de entrada que aparece en los conjuntos SIGUIENTES.

4.2. Análisis sintáctico: una función por no terminal

Cada regla de la gramática se volvió una función, y cada alternativa se decide con su conjunto de predicción. Este es *fact*, que tiene dos alternativas:

Listing 3: `fact ->menos fact | unidad` en `sintactico.py`.

```
def fact(self):
    if self.token.clase == "menos":        # PRED = { - }
        signo = self.emparejar("menos")
        return ("neg", self.fact(), signo.linea)
    if self.token.clase in PRED_UNIDAD:    # PRED = PRIMEROS(unidad)
        return self.unidad()
    self.error_sintaxis("un número, una variable, '(', '|' o una función")
```

Los terminales se consumen con `emparejar`, que es la única función que avanza sobre la entrada:

```
def emparejar(self, esperado):
    if self.token.clase != esperado:
        self.error_sintaxis(NOMBRES.get(esperado, esperado))
    actual = self.token
    self.pos += 1
    return actual
```

Y el análisis termina comprobando que no sobre nada: si después de la última sentencia todavía hay tokens, es error, aunque todo lo anterior estuviera bien.

4.3. Asociatividad: el acumulador

Aquí está el detalle que más cuidado pide. Al quitar la recursión izquierda, la gramática quedó repitiendo por la derecha, pero la resta no es asociativa por la derecha:

$$\underbrace{(4-5)-10}_{-11} \neq \underbrace{4-(5-10)}_{9}$$

La transformación de la gramática no decide cómo evaluar: eso lo deciden las acciones semánticas. Por eso *expr'* recibe un acumulador con lo que ya se construyó a la izquierda y lo va envolviendo en cada vuelta:

```
def expr_r(self, acc):
    if self.token.clase in PRED_EXPR_R:    # PRED = { +, - }
        operador = self.token
        self.emparejar(operador.clase)
        derecha = self.term()
        nuevo = ("bin", OPERADORES[operador.clase], acc, derecha, operador.
    linea)
        return self.expr_r(nuevo)
    return acc                             # expr' -> epsilon
```

Con `4 - 5 - 10` el acumulador pasa por `4`, luego `(4 - 5)` y luego `((4 - 5) - 10)`, que da -11. Hay una prueba automática exactamente para eso.

4.4. Traza del análisis

Para poder revisar las decisiones agregué la opción `--traza`, que muestra qué regla se aplicó en cada paso y qué quedaba por leer:

Listing 4: Traza de `resta = 4 - 5 - 10;`, con la opción `--traza`.

```
Paso  Función / acción           Entrada pendiente
1     prog -> sents              resta = 4 - 5 - 10 ; $
2     sents -> sent sents        resta = 4 - 5 - 10 ; $
3     sent -> ID = expr ;        resta = 4 - 5 - 10 ; $
4     emparejar resta            = 4 - 5 - 10 ; $
5     emparejar =                4 - 5 - 10 ; $
6     expr -> term expr'         4 - 5 - 10 ; $
7     term -> fact term'         4 - 5 - 10 ; $
8     fact -> unidad             4 - 5 - 10 ; $
9     unidad -> NUM              4 - 5 - 10 ; $
10    emparejar 4                - 5 - 10 ; $
11    term' -> ε                 - 5 - 10 ; $
12    expr' -> - term expr'      - 5 - 10 ; $
13    emparejar -                5 - 10 ; $
14    term -> fact term'         5 - 10 ; $
15    fact -> unidad             5 - 10 ; $
16    unidad -> NUM              5 - 10 ; $
17    emparejar 5                - 10 ; $
18    term' -> ε                 - 10 ; $
19    expr' -> - term expr'      - 10 ; $
20    emparejar -                10 ; $
21    term -> fact term'         10 ; $
22    fact -> unidad             10 ; $
23    unidad -> NUM              10 ; $
24    emparejar 10               ; $
25    term' -> ε                 ; $
26    expr' -> ε                 ; $
27    emparejar ;                $
28    sents -> ε                 $
29    fin de entrada: aceptada   $
```

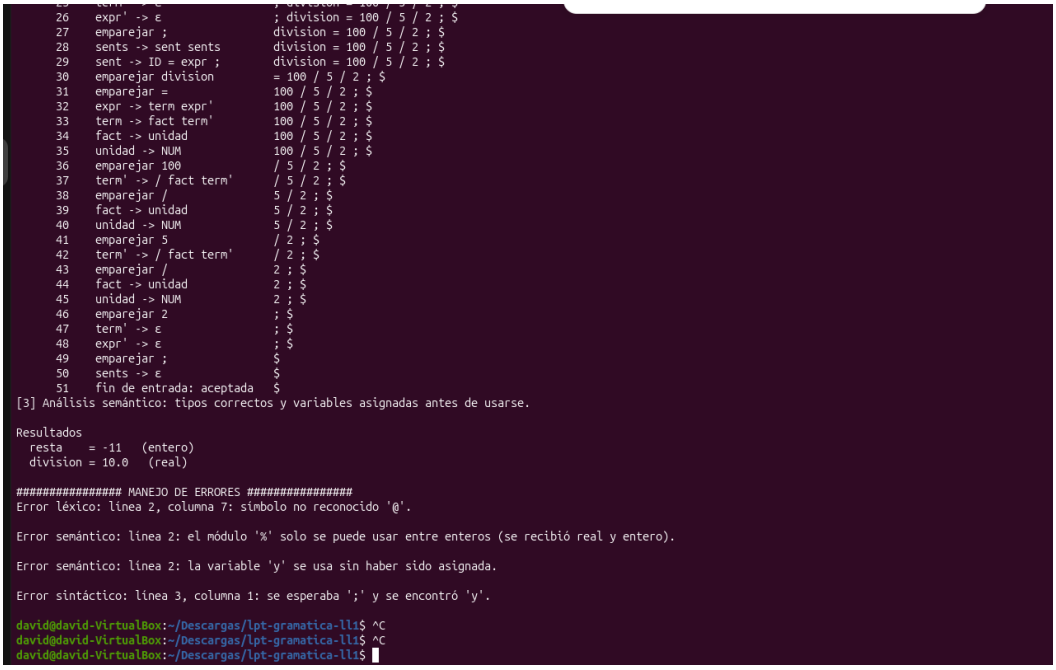


Figura 3: Final de la traza, aceptación de la entrada y los cuatro errores de las tres fases.

Se ve el patrón del descenso recursivo: baja hasta *unidad* para tomar el `4`, sube cerrando *term'* con ε, y solo entonces *expr'* toma el primer `-`. Al final la entrada queda en `$` y se acepta.

4.5. Análisis semántico

La gramática dice qué está bien formado, no qué tiene sentido. `x = y + 1;` es sintácticamente perfecto aunque `y` no exista, y no hay forma de expresar eso en una gramática libre de contexto. Por eso `semantico.py` recorre el árbol con una tabla de símbolos y revisa:

- que una variable esté asignada antes de usarse;
- los tipos: `+`, `-` y `*` dan entero solo si los dos operandos son enteros; `/` siempre da real; `%` exige enteros de los dos lados; `sin`, `cos` y `tan` devuelven real; `abs` y `|x|` conservan el tipo;
- que no se divida entre la constante cero.

```
Error semántico: línea 2: la variable 'y' se usa sin haber sido
    asignada.
Error semántico: línea 2: el módulo '%' solo se puede usar entre
    enteros (se recibió real y entero).
```

Después, una segunda pasada evalúa el programa y llena la tabla con el valor de cada variable.

5. Pruebas de implementación

En `test_implementacion.py` hay 36 pruebas automáticas con `unittest`:

Cuadro 3: Grupos de pruebas.

Grupo	Pruebas	Qué revisa
Léxico	6	tokens, comentarios, reales, símbolos inválidos
Sintáctico	12	precedencia, asociatividad, paréntesis, menos unario, traza, tokens sobrantes y errores
Semántico	6	tipos, variable sin asignar, módulo con reales, división por cero
Funciones	3	`sin`, `cos`, `tan` y funciones anidadas
Gramática	6	LL(1), sin recursión izquierda, sin prefijos comunes, PRIMEROS, SIGUIENTES y predicción
Transformaciones	2	los dos borradores sí tienen conflictos
Ejemplos	2	los archivos de `ejemplos/` dan el resultado y el error esperados

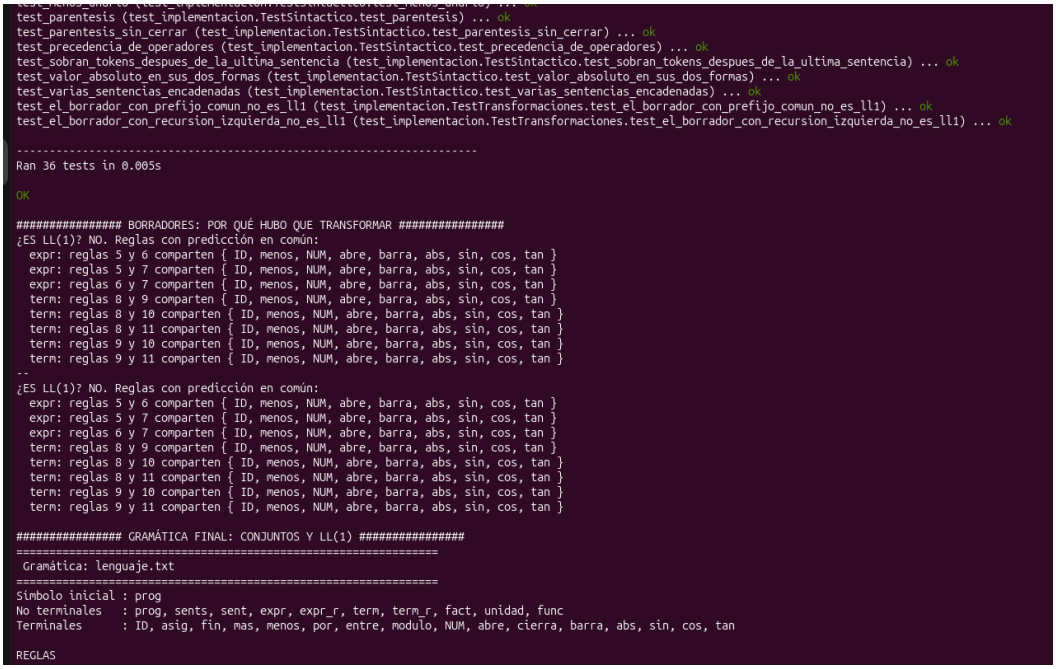


Figura 4: Las 36 pruebas y los conflictos que reportan los dos borradores.

Las dos pruebas de transformaciones son las que dejan constancia de por qué la gramática quedó así: comprueban que los borradores tienen conflicto en *expr* y *term*, y la de la gramática final comprueba que ya no lo tiene. Las de precedencia y asociatividad revisan el otro lado: que además de analizar, el resultado sea el correcto.

Listing 5: Comandos usados.

```
chmod +x run_all.sh
./run_all.sh                                  # pruebas, borradores,
    conjuntos y ejemplos
python3 main.py ejemplos/demo.lp              # análisis completo de un
     programa
python3 main.py ejemplos/demo.lp --traza      # traza del análisis
python3 main.py --conjuntos                   # PRIMEROS, SIGUIENTES,
    predicción y LL(1)
```

6. Ejemplo completo

Listing 6: `ejemplos/demo.lp`.

```
# Programa de prueba del lenguaje

base = 12;
altura = 5;
area = base * altura / 2;

resto = 17 % base;
desfase = |3 - 10|;
negativo = -base + 4;

angulo = 0.7853981634;          # 45 grados en radianes
razon = sin(angulo) / cos(angulo);
mezcla = abs(-2.5) * (base + 1) - tan(0.0);
```

Listing 7: Salida de `python3 main.py ejemplos/demo.lp`.

```
[1] Análisis léxico: 72 tokens reconocidos.
[2] Análisis sintáctico: 9 sentencias, sin errores.
[3] Análisis semántico: tipos correctos y variables asignadas antes de
    usarse.

Resultados
  base     = 12   (entero)
  altura   = 5    (entero)
  area     = 30.0   (real)
  resto    = 5    (entero)
  desfase  = 7    (entero)
  negativo = -8   (entero)
  angulo   = 0.785398   (real)
  razon    = 1.0    (real)
  mezcla   = 32.5   (real)
```

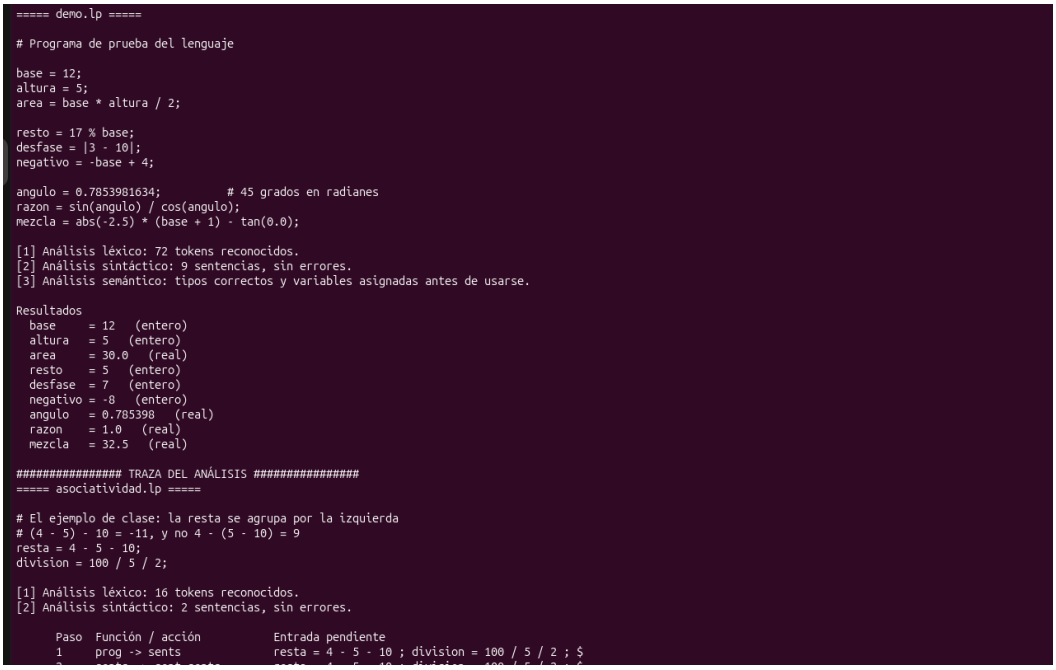


Figura 5: Ejecución de `demo.lp` en Ubuntu.

Se puede comprobar a mano: `area` es $12 \times 5/2 = 30$ y queda real porque hay una división; `resto` es 17 mód $12 = 5$ y queda entero; `desfase` es $|3 - 10| = 7$; y `razon` es $\tan(45°) = 1$, calculado como seno sobre coseno.

7. Conclusiones

- La gramática no quedó LL(1) de entrada. Quedó LL(1) después de aplicar las transformaciones sobre el borrador natural, y eso solo se puede afirmar recalculando los conjuntos de predicción sobre la versión final.
- Los conjuntos de predicción no son un ejercicio aparte: son literalmente las condiciones de los `if` del analizador. Por eso el código no necesita retroceso.
- Quitar la recursión izquierda arregla el análisis pero no la evaluación. La asociatividad por la izquierda hay que devolverla con el acumulador, y sin eso `4 - 5 - 10` daría 9 en vez de -11.
- Separar las tres fases deja cada error donde corresponde: un símbolo raro lo ve el léxico, un `;` faltante lo ve el sintáctico y una variable sin asignar solo la puede ver el semántico.